

1) Define a structure in c and cpp?

Structure in c

A structure in c is a user-defined data type that allows the programmer to group variables of different data types under a single name.

It is mainly used to represent a record containing related data.

Ex

struct Student

```
{  
    int rno;  
    char name[20];  
    float marks;  
};
```

Structure in cpp

A structure in c++ is a user defined data type that groups variables of different data types into a single unit. Unlike c, a structure in c++ can also contain member functions, Constructors and other features of object-oriented programming.

Ex . Struct Student

```
{  
    int rno;  
    char name[20];  
    float marks;  
    void show ( )  
    {  
    }  
};
```

~~Note~~
Note

The main diff in c and cpp: In c structure can contain only data members, but in cpp structure contains both data members & member functions.

c prog

```
struct student
{
    int rno;
    int age;
    float marks;
    char name [10];
};
```

```
int main()
{
    printf("Size of student
    structure = %d bytes",
    sizeof(struct student));
    return 0;
}
```

o/p

Size of Student structure = 22 bytes
manually Calculation: $4 + 4 + 4 + 10 = 22$ bytes

cpp prog

```
#include <iostream>
using namespace std;
struct Student
{
    int rno;
    char name [20];
    float marks;
};
```

```
int main()
```

```
{
    cout << "Size of student
    structure = " << sizeof
    (Student) << "bytes";
    return 0;
}
```

o/p

Size of Student structure = 28 bytes

manually Calculation :- $4 + 20 + 4 = 28$ bytes

(structure padding)

Comparison of Manual Calculation & Size of ()
function:-

~~Comparison of Manual~~

Manual Calculation

calculates the structure size by adding the size of all data members

Does not consider padding bytes

May not give the extra memory used

Ex: $4 + 4 + 4 + 10 = 22$ bytes

Size of() Function

calculates the actual memory occupied by the structure

Includes padding bytes added by the compiler

Gives the exact size allocated in memory

Ex: size of (struct Student) = 24 bytes (with padding)

Manual Calcul = Sum of sizes of all data members

$\Rightarrow$ sizeof() Function = Actual memory occupied by the structure, including padding

$\Rightarrow \therefore$, the Result of sizeof() may be equal to or greater than the manual calculation